CARNEGIE MELLON UNIVERSITY AFRICA

Upanzi Network

AI Coding Agents and Software Development in MOSIP

Code volume, code quality, and code-review practice following AI adoption

Working research report

October 2026

Repositories	20 MOSIP core-service repositories
Repository-months	326
Commits	6,786
Pull requests	5,603
Observation window	January 2025 to August 2026

Revised working draft for research discussion. Observational estimates; not a causal estimate.

Abstract

AI coding assistants may affect code production and code review through different mechanisms. This report examines twenty core-service repositories in the MOSIP digital public infrastructure project over January 2025 to August 2026. It distinguishes adoption of an AI code-review bot from later adoption of coding agents and revises the initial descriptive analysis using interrupted time-series (ITS) regression with repository fixed effects and repository-clustered standard errors.

The descriptive comparison found that the median repository grew in code size by 11%, while absolute cognitive complexity rose by 2% and code smells by 5%. These raw changes do not establish deterioration caused by AI. The ITS analysis estimates pre-adoption growth of 1.28% per month for code size, 1.46% for cognitive complexity, and 1.53% for code smells. Once these pre-existing trajectories are modelled, estimated level changes are small and negative, and no post-adoption slope change is statistically distinguishable from zero. The largest reported production discontinuity is a 255.4% level increase in lines added per month, without a statistically significant level change in commit frequency. This result requires sensitivity checks for bulk imports and influential repository-months before being interpreted as evidence of larger commits.

For review practice, AI review coverage rose from zero to 18.3% of pull requests in the descriptive comparison. Human review coverage fell from 72.3% to 61.5%, while human reviews per pull request increased. However, the estimate for human-review coverage was already declining before adoption, and its ITS level and slope changes were not statistically significant. The evidence is observational, the post-adoption window is short, and the estimates should not be interpreted as causal effects of AI adoption.

Contents

1 Introduction and Motivation **3**
1.1 Research Questions . . . 3

2 Methods **3**
2.1 Study Population and Repository Selection . . . 3
2.2 Observation Window and Data Sources . . . 5
2.3 Detection of AI Adoption . . . 5
2.4 Event-Time Construction . . . 5
2.5 Interrupted Time-Series Specification . . . 6

3 Results **7**
3.1 RQ1: Code Volume . . . 7
3.2 RQ2: Code Quality . . . 7
3.3 Interrupted Time-Series Estimates . . . 8
3.4 RQ3: Code-Review Practice . . . 9

4 Discussion **11**

5 Threats to Validity **11**
5.1 Near-Simultaneous Adoption . . . 12
5.2 Linear Pre-Trend Assumption . . . 12
5.3 Branch and Outcome Scope . . . 12
5.4 Review-Event Data Reconciliation . . . 12
5.5 Outcome Scale and Linear Trend . . . 12
5.6 Short Post-Adoption Window . . . 12

5.7 Small Treatment Dose 12
5.8 Unbalanced Panel 13
5.9 Single Organisation 13

6 Conclusion and Next Steps **13**

A Supplementary Tables and Definitions **14**
A.1 Operational Definitions 14
A.2 Interpretation of the ITS Results 14

B Figure Reading Guide **15**

Author Sign-off **17**

1. Introduction and Motivation

AI coding assistants are now part of mainstream software engineering workflows. Recent empirical work has reported evidence that adoption of coding agents can increase short-term development velocity while being accompanied by increases in software complexity. In particular, He et al. [1] study Cursor adoption across open-source projects using a difference-in-differences design and report an increase in velocity alongside worsening complexity growth.

This report asks a narrower question: what changed within a single organisation when AI assistance was introduced into an established software engineering process? The setting is the MOSIP digital public infrastructure project, an open-source foundational identity platform designed to support countries in building and managing national ID systems [6]. Its core services are developed and maintained within an engineering organisation. This makes the setting relevant for studying changes in code volume, measured software quality, and review practice in a production-oriented codebase.

A particularly useful feature of the MOSIP setting emerged during data collection. The organisation adopted two different classes of AI tools at different times. An AI code-review system, CodeRabbit, was introduced first, while AI coding agents were introduced several months later. These interventions should not be treated as the same treatment. A review bot changes the review workflow, whereas a coding agent changes the process by which code is produced.

The analysis therefore separates the two interventions.

1.1 Research Questions

The study addresses three research questions.

RQ1 Volume.
Did the amount of code, and the rate at which it is produced, change after AI coding agents were adopted?

RQ2 Quality.
Did measured internal code-quality indicators, including complexity, code smells, duplication, bugs, vulnerabilities, and technical debt, change after adoption?

RQ3 Review practice.
Did adoption of an AI code-review bot substitute for human review effort, or add to it?

The questions are intentionally descriptive. The primary analysis compares each repository with itself before and after adoption. Because adoption was nearly simultaneous across repositories, the design does not identify a clean causal effect of AI adoption independent of calendar time or organisation-wide changes.

2. Methods

2.1 Study Population and Repository Selection

The MOSIP GitHub organisation contained 109 public repositories at the time of collection, of which 97 were neither archived nor forks. Repository stars were not used as the selection criterion because the organisation

contains repositories with heterogeneous roles and because star counts do not measure analysable source volume.

Instead, repositories were ranked using JVM source volume obtained through the GitHub languages API. Repositories whose names indicated mocks, test harnesses, deployment or infrastructure, configuration, or documentation were excluded. This produced 36 candidate core services. The twenty repositories with the largest JVM source volumes were selected for the main analysis, representing 47.2 MB of JVM source code.

Checkout size was not used because it can be dominated by history and binary artifacts. For example, the packet-manager repository occupies approximately 188 MB on disk but contains only about 0.51 MB of Java source.

Table 1: Repository selection funnel for the AI-impact analysis

Selection stage	Repositories	Rule or description
Public repositories	109	All public repositories in the MOSIP GitHub organisation at collection time.
Eligible repositories	97	Excluded archived repositories and forks.
Candidate core services	36	Excluded repositories identified as mocks, test harnesses, infrastructure/deployment, configuration, or documentation.
Analysed repositories	20	Selected the twenty candidates with the largest JVM source volumes.
JVM source represented	–	47.2 MB across the selected twenty repositories.

The twenty repositories in this study were selected by source volume and repository role. They are not the same sample as the fifteen core repositories in the earlier MOSIP community-health report. The two studies answer different questions and use different inclusion criteria: this report studies coding activity and code-quality trajectories in twenty high-source-volume repositories, whereas the reference report examines community and pull-request health in fifteen core repositories. Their repository counts and metrics should therefore not be combined as if they described one common sample.

Table 2: The 20 repositories included in the AI-impact sample

Repository	Repository
`admin-services`	`android-registration-client`
`bio-utils`	`commons`
`esignet`	`esignet-signup`
`gist`	`id-authentication`
`id-repository`	`imagedecoder`
`keymanager`	`mosip-openid-bridge`
`nfiq`	`packet-manager`
`partner-management-services`	`pre-registration`
`print`	`registration`
`registration-client`	`resident-services`

The per-repository diagnostic figure labels `gist` as having no detected AI adoption and no SonarQube data. Its inclusion in the plotted 20-repository set should not be interpreted as evidence that it contributes usable observations to every analysis; outcome-specific inclusion depends on data availability and the stated eligibility rules.

2.2 Observation Window and Data Sources

Data were collected in September 2026 for January 2025 through August 2026. Three independent data sources were used.

Table 3: Data sources and coverage

Source	Measures extracted	Volume
Git history	Commits, lines added and removed, contributors	6,786 commits; 3.19M lines added
GitHub API [4, 5]	Pull requests, reviews, issues	5,603 PRs; 17,090 reviews; 1,222 issues
SonarQube 26.8	ncloc, cognitive complexity, code smells, technical debt, bugs, vulnerabilities, duplication	286 snapshots; 285/326 repository-months (87%)

SonarQube reports were reconstructed historically by checking out the last commit of each month and re-analysing the project in full on a dedicated Linux host. The measured indicators follow SonarQube's metric definitions and issue categories [3]. This produced 286 independent analyses. Months in which no code landed on the integration branch generally produced no snapshot, accounting for most of the missing 13%.

The analysis was run on source without compilation, matching the procedure used in the comparison study by He et al. [1].

The revised primary analysis aligns velocity and quality measures to the same development tree. MOSIP uses a gitflow process in which `master` functions as a release branch rather than a complete record of day-to-day development. For example, the `registration-client` repository had 2 commits on `master` compared with 39 on `develop` in the examined comparison. Velocity measures for the ITS analysis were therefore reconstructed from `origin/develop`, which is also the branch used for the corresponding SonarQube snapshots. This replaces the earlier all-branches velocity analysis for the ITS results and improves comparability between code activity and code-quality measures. Repository and observation-window counts reported for the broader collection should not be read as a claim that every model outcome has a complete observation in every month.

2.3 Detection of AI Adoption

AI adoption was detected from three independent signal families:

1. commit-message trailers and identities, including `Co-Authored-By: Claude` and `cursoragent`;
2. AI-agent configuration files committed to repositories, including `CLAUDE.md`, `.cursor/`, and `AGENTS.md`; and
3. bot accounts acting through GitHub.

The resulting detection procedure identified 4,288 AI-attributable actions.

The median interval between review-bot adoption and coding-agent adoption was seven months, with a repository-level range of four to nine months. RQ1 and RQ2 therefore use coding-agent adoption as the event, while RQ3 uses CodeRabbit adoption.

2.4 Event-Time Construction

For each repository i, calendar time was re-indexed relative to its own coding-agent adoption month:

Table 4: Detected AI tools and organisation-wide adoption

Tool	Class	Actions	Adoption
CodeRabbit	Review	4,141	November 2025 (14/19 repositories)
Cursor	Code generation	64	March to August 2026
OpenAI Codex	Code generation	48	April to June 2026
Claude Code	Code generation	35	June to August 2026

$$\tau_{it} = t - a_i, \tag{1}$$

where a_i is the adoption month for repository i. Repository timelines were then pooled by event time τ for descriptive event-study plots.

The main analysis reports raw measures. No normalisation by a repository's pre-adoption baseline and no division by code size are applied in the primary results. Logarithmic axes are used in figures where repositories differ by orders of magnitude, but the underlying values are unchanged.

The within-repository design means that each repository serves as its own comparison unit. Statistical tests are conducted at the repository level rather than at the repository-month level. Each repository contributes one pre-adoption median and one post-adoption median, avoiding an inflated effective sample size caused by serial correlation between months within the same repository.

For RQ1 and RQ2, the pre-adoption period is $\tau \in [-6, -1]$ and the post-adoption period is $\tau \in [+1, +3]$. For RQ3, the post-adoption period extends to $\tau \in [+1, +6]$. The incomplete current month is excluded. Two-sided Wilcoxon signed-rank tests are used for paired repository-level comparisons.

2.5 Interrupted Time-Series Specification

To account for the possibility that codebases were already changing before AI adoption, the revised analysis estimates an interrupted time-series model for each outcome. Segmented regression is a standard approach for estimating level and trend changes around a clearly defined intervention, although its interpretation depends on assumptions about the underlying trend and concurrent events [2]:

$$y_{i\tau} = \alpha_i + \beta_1 \tau + \beta_2 \mathrm{POST}_{i\tau} + \beta_3 (\tau \times \mathrm{POST}_{i\tau}) + \varepsilon_{i\tau}, \tag{2}$$

where $y_{i\tau}$ is the log-transformed outcome for repository i at event time τ, α_i is a repository fixed effect, τ is the month index relative to coding-agent adoption, and $\mathrm{POST}_{i\tau}$ indicates the post-adoption period. The coefficient β_1 estimates the pre-adoption trend, β_2 estimates the immediate level change at adoption, and β_3 estimates the change in slope after adoption. Standard errors are clustered by repository.

The event-time window is $\tau \in [-10, +3]$. A repository is included in a given ITS outcome only if it has at least three pre-adoption observations and at least one post-adoption observation. Thus, the level and slope estimates are informed by repositories that actually straddle the adoption event. For log outcomes, reported percentage changes should be computed as $100(\exp(\beta) - 1)$ when the underlying coefficient is available; the small-coefficient approximation is not appropriate for a reported change as large as 255.4%. The summary available for this revision reports the transformed estimate for lines added per month, but the underlying coefficient and confidence interval should be checked against the model output before publication.

The ITS estimates the change relative to a model-implied counterfactual under continuation of the pre-adoption trend. It does not establish what would have happened absent adoption. Because adoption timing is highly concentrated and calendar-month fixed effects are not included in this specification, concurrent organisation-wide changes remain a threat to interpretation. The reported estimates are therefore observational associations, not identified causal effects.

3. Results

3.1 RQ1: Code Volume

The initial repository-level pre/post comparisons suggested that code size and lines added increased after coding-agent adoption. Those comparisons did not account for the pre-existing trajectory. The revised ITS results provide a more cautious interpretation: code size was already increasing by an estimated 1.28% per month before adoption, and the post-adoption slope change was small and statistically non-significant. The estimated level change was −2.20%.

The clearest discontinuity in the current ITS results is lines added per month, with an estimated +255% level change and no statistically significant increase in commits per month. This is consistent with larger commits rather than more frequent commits. However, the estimate should be checked against bulk imports, influential repository-months, and alternative outcome definitions before being treated as robust evidence.

The original raw event-study and commit-size figures remain useful descriptive context, but they do not adjust for pre-existing trends. Figure 1 shows raw distributions by event time; the commit-size figure distinguishes added lines, deleted lines, and total churn. Readers should not interpret changes between columns as a treatment effect because the set of repositories contributing observations changes over event time.

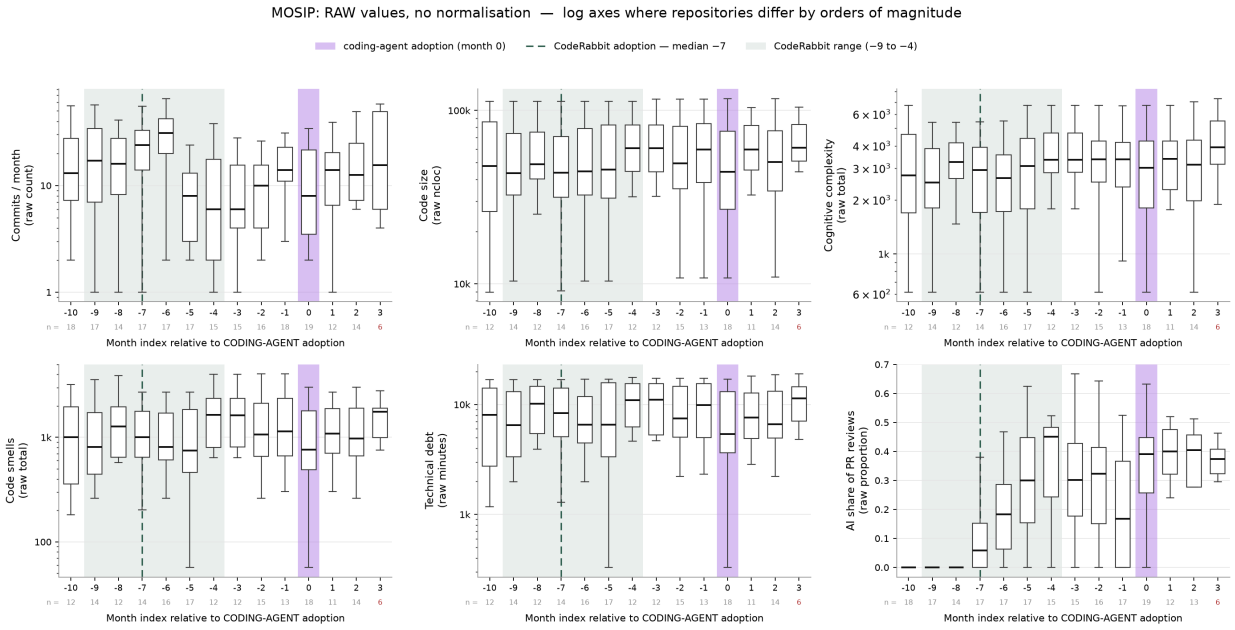


Figure 1: Raw event-time distributions around coding-agent adoption. The plots describe the observed data but do not control for pre-adoption trends or changing repository composition.

3.2 RQ2: Code Quality

The original paired pre/post medians showed increases in absolute cognitive complexity and code smells. These measures, however, naturally tend to increase as a codebase grows. The ITS model addresses the central concern by estimating the pre-adoption trajectory and testing for an additional level or slope change at adoption.

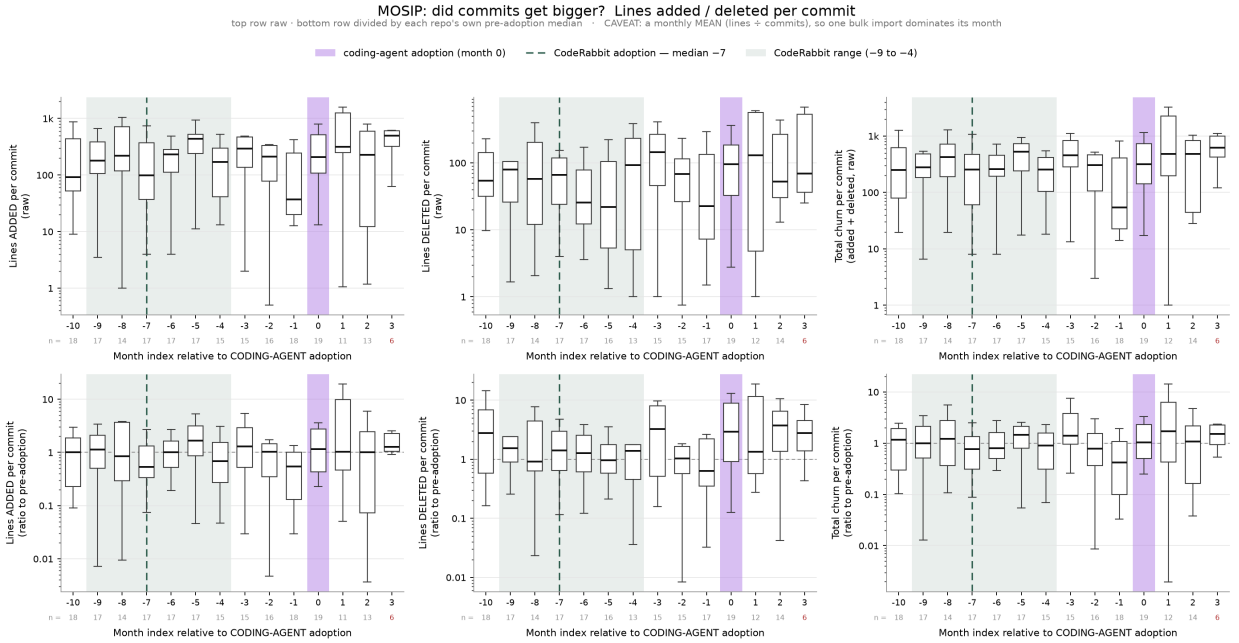


Figure 2: Lines added, lines deleted, and total churn per commit, shown in raw units and relative to each repository's pre-adoption median. The raw monthly mean can be dominated by a bulk import, so per-commit distributions and influence checks are needed.

The pre-adoption trends are positive for code size (+1.28% per month), cognitive complexity (+1.46% per month), and code smells (+1.53% per month). After controlling for those linear trends, the estimated level changes are negative for all three outcomes: −2.20% for code size, −2.58% for cognitive complexity, and −3.58% for code smells. The post-adoption slope changes are small and not statistically significant. Technical debt also has a negative estimated level change (−4.04%), with no significant slope change.

These results overturn the stronger interpretation suggested by the original pre/post ratios. The initial increases of 11% in code size, 2% in cognitive complexity, and 5% in code smells should not be described as effects of AI adoption. The outcomes were already trending upward before adoption, and the ITS estimates do not show a statistically significant acceleration after adoption. This does not establish that AI improved code quality: the negative level estimates are not a randomised causal effect, and the short time window limits the ability to estimate post-adoption slopes precisely.

3.3 Interrupted Time-Series Estimates

Table 5 summarises the revised segmented-regression estimates. The pre-adoption trends for code size, cognitive complexity, and code smells are positive and statistically significant, indicating that these outcomes were already increasing before coding-agent adoption. After accounting for those trends, the estimated level changes are negative for all four quality and size measures, and none of the estimated post-adoption slope changes is statistically distinguishable from zero. These results do not show that AI improved code quality; they show that the raw pre/post increases cannot be attributed to adoption on the basis of this model.

The large estimated level increase in lines added per month should be treated cautiously. Commit counts do not show a statistically significant level increase, which is consistent with larger commits rather than more frequent commits, but it does not establish that mechanism. A bulk import, refactor, generated-code change, or influential repository-month could produce a similar pattern. The underlying monthly data and commit history should be checked before this result is treated as robust.

Table 5: Interrupted time-series estimates. The reported percentages are the supplied transformed estimates for log outcomes. Standard errors and confidence intervals were not included in the analysis summary available for this revision and should be added before publication. Significance: * $p < .05$, ** $p < .01$, *** $p < .001$.

Outcome	Pre-trend (%/mo.)	Level change (%)	Slope change (%/mo.)	Obs.	Repos.
Code size (ncloc)	$+1.28^{***}$	-2.20^{*}	+0.12	181	17
Cognitive complexity	$+1.46^{**}$	−2.58	+0.29	181	17
Code smells	$+1.53^{**}$	-3.58^{**}	−0.16	181	17
Technical debt	+0.82	-4.04^{*}	+0.49	181	17
Commits per month	-8.06^{*}	+27.62	+9.08	181	17
Lines added per month	−9.66	$+255.4^{*}$	+1.67	170	16
PRs opened per month	-4.14^{*}	−17.58	+17.12	212	19

The descriptive pre/post comparison and the ITS answer different questions. The former reports the observed difference across the adoption boundary. The latter estimates a level and slope change relative to the fitted pre-adoption trajectory. The ITS does not include calendar-month fixed effects, so near-simultaneous adoption and other changes occurring around the same time remain plausible alternative explanations.

Figure 3 indexes observations to each repository's own pre-adoption median. It helps visualise within-repository changes but does not substitute for the ITS model. Figure 4 displays repository-specific code-size and cognitive-complexity trajectories and shows why a pooled average can conceal heterogeneous paths.

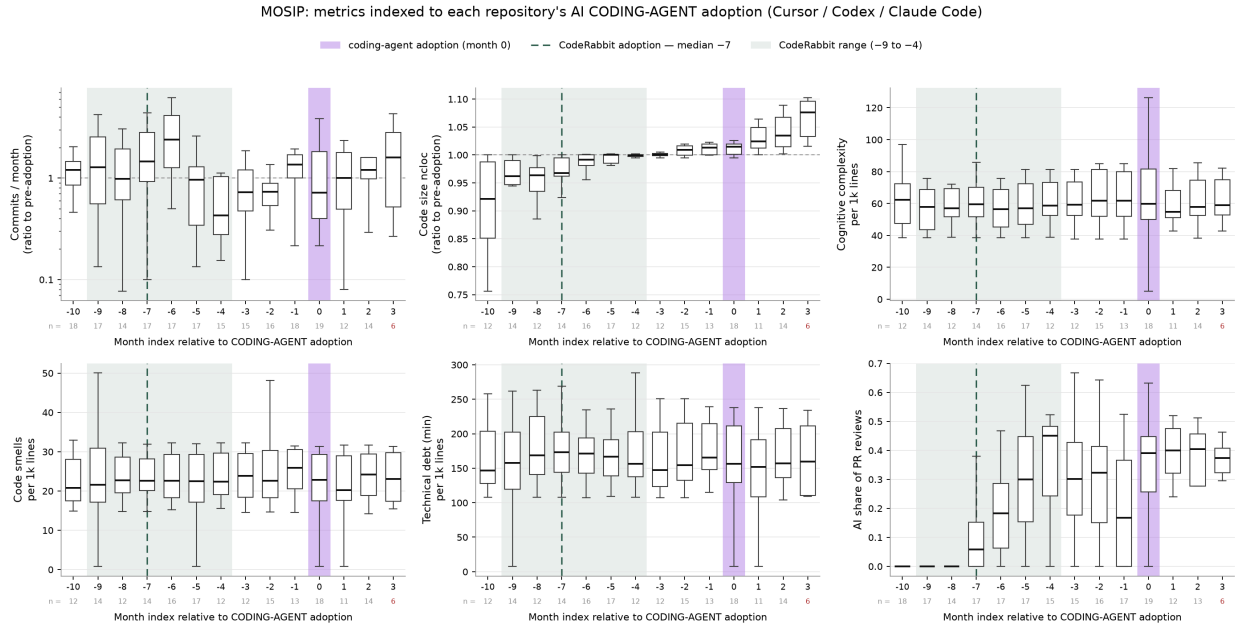


Figure 3: Event-time measures indexed to each repository's pre-adoption median. Normalisation improves visual comparability across repositories but does not control for time trends.

3.4 RQ3: Code-Review Practice

RQ3 is evaluated against AI review-bot adoption, using pre-adoption months $\tau \in [-6, -1]$ and post-adoption months $\tau \in [+1, +6]$. Review coverage is the fraction of pull requests receiving at least one review of a given type. Review intensity is the number of review submissions per pull request. These measures should not be

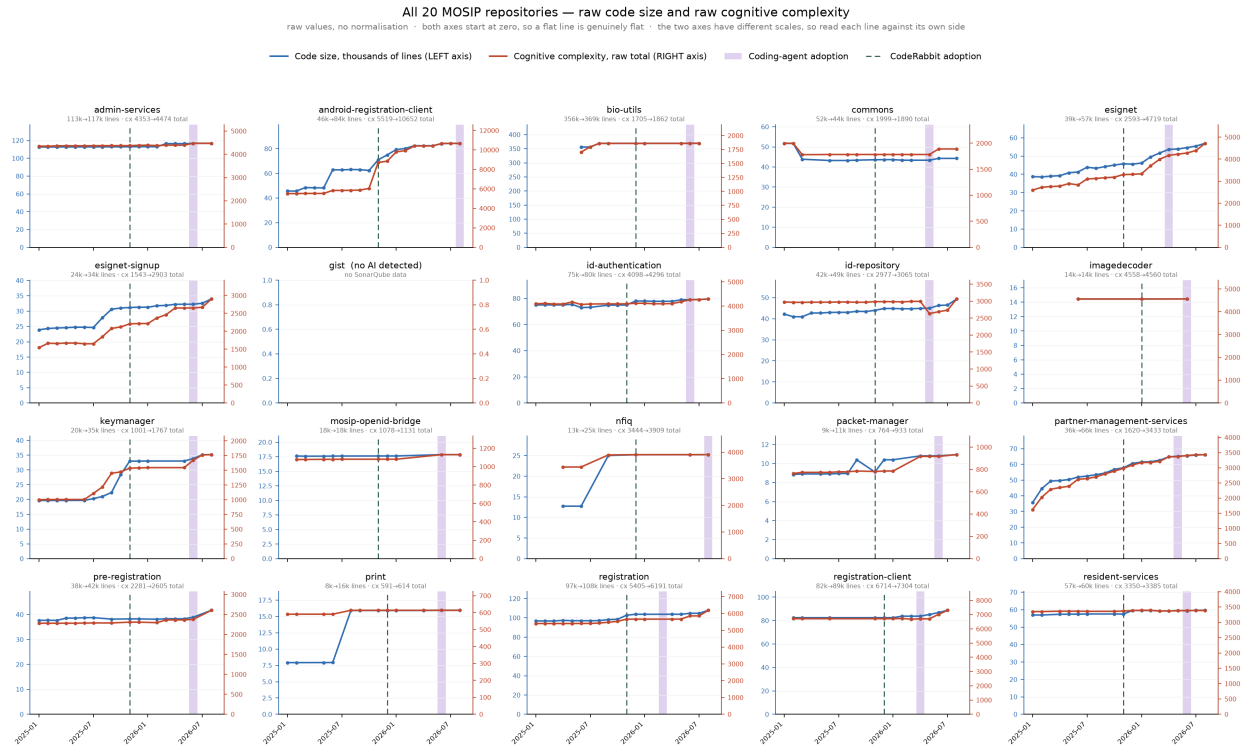


Figure 4: Repository-level trajectories for code size and cognitive complexity. The lines show heterogeneous changes across repositories and are descriptive rather than causal estimates.

conflated: more review submissions per PR can coexist with fewer PRs receiving human review.

The repository-level descriptive comparison shows that overall review coverage fell from 72.3% to 67.9%, a change that was not statistically significant ($p = .241$). Human-review coverage fell from 72.3% to 61.5% (−10.8 percentage points, $p = .0012$), while AI-review coverage rose from zero to 18.3% ($p < .0001$). Human review submissions per PR increased, but the supplied revision contains two different pre-adoption figures: 1.43 in the earlier report and 1.27 in the latest analysis notes. This value must be reconciled against the underlying repository-level table before the paper is finalised; Table 6 uses 1.27 provisionally to match the latest supplied analysis notes.

Table 6: Review coverage and intensity before and after AI review-bot adoption. Repository-level paired comparison, $n = 19$. The human-review submissions-per-PR baseline is provisional pending reconciliation with the earlier report.

Measure	Pre	Post	Change	p
PRs receiving any review	72.3%	67.9%	−4.4 pp	.241
PRs receiving human review	72.3%	61.5%	−10.8 pp	.0012
PRs receiving AI review	0.0%	18.3%	+18.3 pp	< .0001
Human reviews per PR	1.27	1.80	+0.53	.011

The ITS analysis qualifies the raw decline in human-review coverage. Human-review coverage was already declining at approximately −0.70 percentage points per month before adoption. The estimated level change was +5.9 percentage points and the slope change was −2.1 percentage points per month; neither was statistically significant. Therefore, the descriptive fall in human-review coverage should not be presented as an effect caused by CodeRabbit.

The supplied review-submission counts also require reconciliation before reporting pooled actor shares.

The latest notes give a total of 15,824 submissions, 10,638 human submissions, 4,137 AI submissions, and 9 non-AI automated submissions. These categories sum to 14,784, not 15,824. The current report therefore does not use those pooled counts or their implied percentages. Recompute the denominator and actor classification from the raw review-event data before publication.

Finding 3. AI-review coverage increased from zero to 18.3% of PRs, while human-review coverage fell in the descriptive comparison. Human reviews per PR increased, suggesting that review activity may have become more concentrated among PRs that received human attention. However, overall coverage did not change significantly, and the ITS did not identify statistically significant level or slope changes in human-review coverage. The evidence does not establish that CodeRabbit caused the decline in human coverage or that it expanded human review effort overall.

The figure intended to show coverage, review intensity, and reviewer composition was not present in the source package used to build this report. It has not been fabricated or inserted as a placeholder graphic. Add the verified figure only after confirming that its panels and labels match the underlying review-event analysis.

4. Discussion

The revised analysis changes the interpretation of the initial descriptive findings. Code size, cognitive complexity, and code smells were already increasing before coding-agent adoption. After modelling those pre-adoption trends, the ITS estimates show negative level changes and no statistically significant slope changes for the reported quality outcomes. The raw pre/post increases should therefore not be presented as evidence that AI coding agents caused code growth or worsening quality. A more precise conclusion is that the observed changes are consistent with the pre-existing trajectory, and this analysis detects no statistically significant positive adoption-associated change in the measured quality outcomes.

The most notable estimated discontinuity is a 255.4% increase in lines added per month, without a statistically significant level change in commit counts. This is consistent with larger commits rather than more frequent commits, but the mechanism is not established. Bulk imports, refactors, generated code, or influential repository-months could produce a similar pattern. The estimate should be rechecked using commit-level distributions and sensitivity analyses before being highlighted as a substantive result.

Review outcomes also require a distinction between coverage and intensity. The fraction of PRs receiving human review declined in the descriptive comparison, while the number of human review submissions per PR increased. The review coverage decline was already underway before CodeRabbit adoption, and the ITS level and slope changes were not significant. The current evidence does not establish that CodeRabbit caused the coverage decline or increased human review effort overall. In addition, pooled review-submission counts and the pre-adoption reviews-per-PR figure remain subject to data reconciliation.

The figures have different inferential roles. Raw event-time plots describe distributions and repository composition; normalised plots make relative within-repository movement easier to see; per-repository plots reveal heterogeneity; and the ITS table models level and slope changes relative to a fitted pre-adoption trend. An ITS figure with observed data and fitted trajectories should be added once its source data and construction are available. Even a correctly specified ITS remains vulnerable to concurrent events, model misspecification, and the short post-adoption period.

5. Threats to Validity

5.1 Near-Simultaneous Adoption

Fourteen of nineteen repositories adopted CodeRabbit in a single month. Adoption timing is therefore highly correlated with calendar time. The same issue applies to the later coding-agent rollout. In a standard two-way fixed-effects event-study specification, treatment timing that is effectively common across units is not separately identifiable from calendar-time effects.

5.2 Linear Pre-Trend Assumption

The ITS model assumes a linear pre-adoption trend and extrapolates that trend into the post-adoption period. Nonlinear trends or other concurrent changes can bias the estimated level and slope changes. Sensitivity checks should examine alternative windows and functional forms.

5.3 Branch and Outcome Scope

The revised ITS aligns velocity measures to `origin/develop`, the development branch also used for quality snapshots. This improves consistency with the earlier all-branch analysis, but the report should retain exact per-outcome sample sizes and document missing observations so readers can determine which repositories identify each estimate.

5.4 Review-Event Data Reconciliation

The supplied review summary contains an inconsistency between the total number of review submissions and the reported human, AI, and non-AI automated categories. The pre-adoption human-reviews-per-PR figure also differs between the earlier draft and the latest revision notes. These quantities must be recomputed from the raw event data with explicit definitions of a review submission, reviewer class, PR denominator, and observation window. Until then, the review coverage comparisons and the pooled actor counts should be treated as separate evidence, and the inconsistent pooled totals should not be reported.

5.5 Outcome Scale and Linear Trend

Absolute complexity and code-smell counts can rise simply because a codebase grows. The ITS analysis addresses the pre-existing time trend, but it does not directly estimate quality conditional on code size. A size-conditioned model or a rate-based outcome would provide a complementary test. The ITS also assumes that the pre-adoption trend would have continued linearly in log terms absent adoption. This assumption is untestable with the current data, and the short post-adoption window gives limited information about slope changes.

5.6 Short Post-Adoption Window

Coding agents arrived in the middle of 2026, and the dataset ends in August 2026. By three months after adoption, only six repositories remain in the pooled event-time panel. Longer-term maintainability effects are therefore not yet observable.

5.7 Small Treatment Dose

Only 147 commits are directly attributable to coding agents across 326 repository-months. The observed treatment exposure is consequently limited.

5.8 Unbalanced Panel

Repositories enter and leave the event-time columns, and only five repositories appear at both $\tau = -10$ and $\tau = +3$. The within-repository tests remain based on paired observations, but raw comparisons across event-time columns can be misleading when the composition of repositories changes.

5.9 Single Organisation

MOSIP has its own engineering culture, review norms, repository structure, and gitflow process. Results from this setting should not be generalised to other organisations without replication.

6. Conclusion and Next Steps

The revised ITS analysis shows why pre/post comparisons alone were insufficient. Code size, cognitive complexity, and code smells were already growing before coding-agent adoption, at approximately 1.3% to 1.5% per month. Once these trends are included, estimated level changes are negative and post-adoption slope changes are not statistically significant. The current evidence is consistent with continuation of pre-existing trends and does not establish that AI coding agents caused the original increases in absolute code size, complexity, or smells.

The largest reported production discontinuity is a 255.4% level increase in lines added per month, without a statistically significant level increase in commit counts. Larger commits are one possible explanation, but the result requires checks for bulk imports, refactors, generated code, and influential repository-months. For review practice, AI-review coverage rose from zero to 18.3% of PRs. Human-review coverage fell in the descriptive comparison, but it was already declining before adoption and the ITS level and slope changes were not significant. Pooled review-event totals and the human reviews-per-PR baseline must be reconciled before publication.

The next steps are to:

1. validate the review-event denominator, reviewer categories, and pre-adoption human-reviews-per-PR value against the raw data;
2. run robustness checks for the lines-added discontinuity, including excluding bulk imports and influential repository-months;
3. add standard errors or confidence intervals and exact outcome-specific sample sizes to the ITS table;
4. add the ITS and review figures after verifying their underlying data and panel definitions;
5. test alternative ITS windows and trend specifications, and consider calendar-time sensitivity analyses; and
6. extend the observation period to improve estimation of post-adoption slope changes.

These analyses should be described as observational interrupted time-series estimates, not causal effects. A matched external control cohort would strengthen the design by helping separate AI adoption from concurrent organisation-wide changes.

Data Availability

The underlying descriptive figures and paired pre/post statistics are reported as reproducible from per-repository reports and the pooled `panel_ai_impact.csv`, using scripts in `_tools/analysis/`. The revised ITS results use repository fixed effects and standard errors clustered by repository. The analysis summary supplied for this revision did not include full coefficient standard errors or confidence intervals. Review-event totals and one review-intensity baseline also remain unreconciled. These items should be resolved from the analysis outputs before formal circulation.

A. Supplementary Tables and Definitions

A.1 Operational Definitions

Table 7 documents the main outcomes and the intended interpretation of each measure. These definitions are included to make the analysis easier to audit and to reduce the risk of conflating code volume, code quality, and review activity.

Table 7: Operational definitions of the principal measures

Measure	Operational definition	Interpretation and caveat
Code size (ncloc)	Non-comment source lines reported by SonarQube.	Measures codebase size, not quality by itself; it can rise as functionality is added.
Cognitive complexity	SonarQube cognitive-complexity measure.	A proxy for the difficulty of understanding control flow; absolute totals can rise with codebase size.
Code smells	Count of maintainability issues identified by SonarQube.	Not equivalent to defects; absolute counts depend on code volume and rule configuration.
Technical debt	SonarQube estimated remediation effort for identified maintainability issues.	An estimate based on configured rules, not a direct measure of realised maintenance cost.
Commits per month	Number of commits on the `origin/develop` branch in each repository-month.	Measures commit frequency on the development branch, not work across every branch.
Lines added per month	Sum of added lines from commits on `origin/develop` in each repository-month.	Can be strongly influenced by bulk imports, generated files, and large refactors.
Review coverage	Fraction of PRs receiving at least one review from the relevant reviewer class.	Measures breadth of coverage, not the number or depth of review submissions.
Reviews per PR	Number of review submissions divided by the number of PRs in the defined window.	Measures intensity; a higher value can coexist with lower review coverage.

A.2 Interpretation of the ITS Results

The reference report, *How Open Source is MOSIP?*, uses community-health and pull-request tables that are not directly applicable to this coding-agent study. Accordingly, this revision adds tables that document this report's sample, measures, and model interpretation rather than copying unrelated contributor-demographic or dependency-management counts into the AI-impact analysis.

Table 8: How to read the interrupted time-series coefficients

Model term	Meaning in this report
Pre-trend (β_1)	Estimated monthly trend before coding-agent adoption.
Level change (β_2)	Estimated immediate discontinuity at the adoption boundary, relative to the fitted pre-adoption trajectory.
Slope change (β_3)	Estimated difference between the post-adoption trend and the pre-adoption trend.
Repository fixed effect (α_i)	Controls for time-invariant differences between repositories.
Repository-clustered standard errors	Allow for within-repository dependence when estimating uncertainty.

B. Figure Reading Guide

Each figure should be interpreted in the main text and in its caption.

- **Raw event study:** reports absolute outcomes by event time. Box heights are influenced by the scale between-repositorys and by which repositories contribute at each event time. It is descriptive and does not control for pre-trends.
- **Commit-size view:** reports added lines, deleted lines, and total churn, in raw units, and relative to each repository's pre-adoption median. Monthly means can be dominated by bulk imports.
- **Normalised event study:** expresses outcomes relative to each repository's own pre-adoption median. This makes relative movement easier to compare, but does not estimate a counterfactual.
- **Per-repository trajectories:** shows that code size and complexity do not move identically in every repository. Heterogeneity should not be hidden by a single pooled summary.
- **ITS estimates:** the table reports pre-adoption trends, level changes, and slope changes. The level and slope changes are estimated relative to the fitted pre-adoption trend, not by comparing the post period with the final pre-adoption observation. An ITS plot is not included in this package because the corresponding verified image was not supplied.
- **Review coverage and intensity:** coverage measures the fraction of PRs receiving a review; intensity measures reviews per PR. They can move in opposite directions. A review figure is not included in this package because the corresponding verified image was not supplied.

References

[1] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*, pages 181–193. Association for Computing Machinery, 2026. https://doi.org/10.1145/3793302.3793349.

[2] James Lopez Bernal, Steven Cummins, and Antonio Gasparrini. Interrupted time series regression for the evaluation of public health interventions: A tutorial. *International Journal of Epidemiology*, 46(1):348–355, 2017. https://doi.org/10.1093/ije/dyw098.

[3] SonarSource. Metric definitions. SonarQube documentation. https://docs.sonarsource.com/sonarqube-server/latest/user-guide/code-metrics/metrics-definition/.

[4] GitHub. REST API endpoints for pull requests and reviews. GitHub documentation. https://docs.github.com/en/rest/pulls.

[5] GitHub. Repositories REST API endpoints. GitHub documentation. https://docs.github.com/en/rest/repos/repos.

[6] MOSIP. MOSIP documentation: The future of digital identity. Modular Open Source Identity Platform. https://docs.mosip.io/1.2.0.

Author Sign-off

Prepared by: Mboh Bless Pearl N

Affiliation: Upanzi Network, Carnegie Mellon University Africa

Date: 19 October 2026